

Lesson 6: Local Storage for Saving User Data

Mission Brief

Every time a crew member closes the mission log and reopens it, the ship should remember exactly what was written before — nobody wants to re-enter today's readings from scratch. Today you learn **Local Storage**, the browser's built-in way to save small amounts of data on a user's own device so it survives a page refresh, a closed tab, or even a restarted computer.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what Local Storage is, what it's good for, and its key limitations
- Save and retrieve string data with `localStorage.setItem()` and `.getItem()`
- Store and retrieve complex data (arrays/objects) using `JSON.stringify()` and `JSON.parse()`
- Remove individual items or clear all storage with `.removeItem()` and `.clear()`
- Load saved data back into the page automatically when it first loads

What You'll Build

A "Mission Log" app: a textarea where a crew member writes a log entry and a "Save Log" button, where the entry is saved to Local Storage and automatically reloaded and displayed the next time the page is opened — even after fully closing and reopening the browser tab.

Prerequisites

- Lesson 2 (DOM manipulation)
- Lesson 3 (event listeners)
- Lesson 1 (arrays and objects, briefly)

Key Concepts

- Local Storage vs. variables in memory (persistence)
- `localStorage.setItem(key, value)` / `.getItem(key)`
- `localStorage.removeItem(key)` / `.clear()`
- `JSON.stringify()` and `JSON.parse()`
- Storage is per-origin and string-only
- Loading saved state on page load

Concept Explanation

Every variable you've used so far — `let`, `const`, arrays, objects — lives only in the browser's memory for as long as the page stays open. Refresh the page, and it's all gone. That's a real problem for any app where a user expects their data to still be there tomorrow: a to-do list, game high scores, saved preferences, or a mission log.

Local Storage solves this. It's a simple key-value storage system built into every browser, tied to your page's **origin** (protocol + domain + port). Data saved with Local Storage persists even after the tab is closed, the browser is quit, or the computer restarts — it only goes away if the user clears their browser data, you explicitly delete it in code, or (for `file://` pages) storage limitations of that browsing context apply. It is not sent to any server; it lives entirely on the user's own device.

The API is deliberately simple: `localStorage.setItem("key", "value")` saves a string under a key, and `localStorage.getItem("key")` retrieves it (returning `null` if that key doesn't exist). `localStorage.removeItem("key")` deletes one entry, and `localStorage.clear()` wipes everything for that origin.

The catch: **Local Storage only stores strings**. If you try to save an array or object directly, JavaScript will convert it to the unhelpful string `"[object Object]"`. The standard solution is `JSON.stringify(value)`, which converts any JSON-safe JavaScript value (objects, arrays, numbers, strings, booleans) into a JSON-formatted string suitable for storage, and `JSON.parse(jsonString)`, which converts that string back into a real JavaScript value when you read it back out. This "stringify before saving, parse after loading" pattern is used constantly in real apps — you'll use it for the to-do app's entire list of tasks in this course's Project 04.

A well-built app checks Local Storage for existing saved data **as soon as the page loads** (not just when the user clicks something), so that a returning visitor immediately sees their previous state restored, rather than starting from a blank slate every time. This means your data-loading code typically runs once near the top of your script, outside of any button's click handler.

Guided Coding

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Mission Log</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; max-
width: 500px; }
    textarea { width: 100%; height: 100px; font-family: inherit; }
    #saved-msg { color: #4ade80; min-height: 1.2em; }
    #entries li { margin-bottom: 0.5rem; padding: 0.5rem; background: #1e2540; border-radius:
4px; }
  </style>
</head>
<body>
  <h1>Mission Log</h1>
  <textarea id="log-input" placeholder="Write today's log entry..."></textarea>
  <br />
  <button id="save-btn">Save Log Entry</button>
  <button id="clear-btn">Clear All Entries</button>
  <p id="saved-msg"></p>
  <h2>Saved Entries</h2>
  <ul id="entries"></ul>
  <script src="script.js"></script>
</body>
</html>

```

```

// script.js

const STORAGE_KEY = "lumexa-mission-log-entries";

const logInput = document.querySelector("#log-input");
const saveButton = document.querySelector("#save-btn");
const clearButton = document.querySelector("#clear-btn");
const savedMsg = document.querySelector("#saved-msg");
const entriesList = document.querySelector("#entries");

// --- READING SAVED ENTRIES (array of strings), safely ---
function loadEntries() {
  const rawValue = localStorage.getItem(STORAGE_KEY);

  if (rawValue === null) {
    return []; // nothing saved yet – start with an empty log
  }

  try {
    const parsed = JSON.parse(rawValue);
    // Defensive check: make sure what we loaded is actually an array,
    // in case storage was ever corrupted or tampered with manually.
    return Array.isArray(parsed) ? parsed : [];
  } catch (error) {
    console.error("Corrupted mission log data, resetting.", error);
    return [];
  }
}

// --- WRITING ENTRIES BACK ---
function saveEntries(entriesArray) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(entriesArray));
}

// --- RENDERING THE LIST TO THE PAGE ---
function renderEntries(entriesArray) {
  entriesList.innerHTML = ""; // clear and rebuild, a simple safe pattern

  entriesArray.forEach((entryText, index) => {
    const listItem = document.createElement("li");
    listItem.textContent = `Entry ${index + 1}: ${entryText}`;
    entriesList.appendChild(listItem);
  });
}

// --- APP STATE, LOADED FROM STORAGE IMMEDIATELY ON PAGE LOAD ---
let missionLogEntries = loadEntries();
renderEntries(missionLogEntries);

```

```
// --- SAVING A NEW ENTRY ---
saveButton.addEventListener("click", () => {
  const entryText = logInput.value.trim();

  if (entryText === "") {
    savedMsg.textContent = "Cannot save an empty entry.";
    return;
  }

  missionLogEntries.push(entryText);
  saveEntries(missionLogEntries); // persist the whole updated array
  renderEntries(missionLogEntries);

  logInput.value = "";
  savedMsg.textContent = "Log entry saved! Reload the page to confirm it's still here.";
});

// --- CLEARING EVERYTHING ---
clearButton.addEventListener("click", () => {
  missionLogEntries = [];
  localStorage.removeItem(STORAGE_KEY); // or localStorage.clear() to wipe the whole origin
  renderEntries(missionLogEntries);
  savedMsg.textContent = "All entries cleared.";
});
```

Code Walkthrough

- **STORAGE_KEY**: a single constant for the storage key name, used everywhere instead of retyping the string, so a typo can't create two different keys by accident.
- **loadEntries()**: reads the raw string from storage. If nothing was ever saved, `getItem` returns `null`, and the function returns a fresh empty array. Otherwise it `JSON.parse()`s the string, wrapped in a `try / catch` to defend against corrupted data (e.g., if a user manually edited storage in DevTools), and confirms the result is actually an array with `Array.isArray()` before trusting it.
- **saveEntries()**: the mirror operation — takes a real array, converts it with `JSON.stringify()`, and writes the resulting string to storage under the same key, fully overwriting whatever was there before.
- **renderEntries()**: pure DOM-building logic from Lesson 2, kept separate from storage logic — a clean separation of concerns (data vs. display).
- **App startup**: `let missionLogEntries = loadEntries();` runs immediately as the script executes (not inside any click handler), so saved data appears the instant the page loads — this is the key behavior that makes the app feel "persistent."
- **Save handler**: modifies the in-memory array (`.push()`), then calls `saveEntries()` to persist the *entire* updated array back to storage (Local Storage has no concept of "add one item" — you always save the whole value under a key), then re-renders.

- **Clear handler:** resets the in-memory array and removes the stored key entirely, then re-renders an empty list.

Student Mission

1. Add a "Delete" button next to each rendered entry (using event delegation from Lesson 3) that removes just that one entry from `missionLogEntries`, calls `saveEntries()` to persist the change, and re-renders.
2. Add a timestamp to each entry when it's saved (e.g., using `new Date().toLocaleString()`), storing each entry as an object `{ text, timestamp }` instead of a plain string, and update `renderEntries` to display both.
3. Test persistence properly: save two entries, fully close the browser tab (not just refresh), reopen the HTML file, and confirm both entries are still there.

Challenge

Add a per-entry "favorite" toggle (a star icon/button) that flips a `favorite: true/false` field on that entry's object, persists it, and visually highlights favorited entries differently — practicing updating a single field inside an array of objects before re-saving the whole array to storage.

Experiment/Extension

Open DevTools → **Application tab** → **Local Storage** → your page's origin. Save an entry through the UI and watch the raw JSON string appear live in that panel. Try manually editing the value there to invalid JSON (like deleting a closing bracket) and reload the page — confirm your `try / catch` in `loadEntries()` gracefully resets to an empty log instead of crashing the whole page.

Common Mistakes

- Trying to save an array/object directly with `setItem` without `JSON.stringify()`, resulting in the useless string `"[object Object]"` being stored.
- Forgetting `JSON.parse()` when reading it back, then getting confusing bugs when treating a string as if it were an array.
- Assuming `getItem()` returns `undefined` for a missing key — it actually returns `null`, and checking `=== undefined` will not catch that case.
- Only saving on some actions but forgetting others (e.g., saving after adding an entry but forgetting to save after deleting one), causing state to "revert" after a reload.
- Not wrapping `JSON.parse()` in a `try / catch`, so any corrupted stored data crashes the entire page on load instead of failing gracefully.

Debugging Tips

- DevTools → **Application tab** → **Local Storage** is the single best tool for this lesson — you can view, edit, and delete raw stored values directly.
- `console.log(localStorage.getItem(STORAGE_KEY))` at any point tells you exactly what's currently stored, as a raw string.
- If your data "disappears" after reload, confirm you're loading from storage at the *top level* of your script (runs immediately), not only inside a button handler that never got clicked.
- Remember Local Storage is per-origin: data saved while opening `index.html` as a `file://` URL versus via a local server (`http://localhost`) are treated as **different origins** and won't share storage — a common source of "my data vanished" confusion.

Check Your Understanding

1. Why can't you store a JavaScript array directly in Local Storage without conversion?
2. What does `localStorage.getItem()` return for a key that was never set?
3. What is the "stringify before saving, parse after loading" pattern for, in your own words?
4. Why should you load saved data near the top of your script rather than only inside a button's click handler?
5. Why is it good practice to wrap `JSON.parse()` in a `try / catch` when loading from storage?

Mini Quiz + Answer Key

Q1. What must you do before storing an array in Local Storage? A) Nothing, arrays are stored automatically B) Convert it with `JSON.stringify()` C) Convert it to a number D) Local Storage cannot store arrays under any circumstances

Q2. What does `localStorage.getItem("missingKey")` return? A) `undefined` B) An empty string C) `null` D) Throws an error

Q3. Which method removes exactly one key from Local Storage? A) `.clear()` B) `.delete()` C) `.removeItem()` D) `.reset()`

Q4. Where should code that loads saved state typically run? A) Only inside a "Load" button's click handler B) Near the top of the script, so it runs as soon as the page loads C) Inside a `setInterval` D) It doesn't matter

Answer Key: 1-B, 2-C, 3-Open response (see explanation), 4-B

Lesson Recap

You learned what Local Storage is and its limits, how to save and load data with `setItem / getItem`, how to store complex arrays and objects using `JSON.stringify() / JSON.parse()`, how to remove data with `removeItem / clear()`, and how to load saved state automatically when the page first opens so your app truly persists between visits.

Homework

Build a "Favorite Star Systems" widget: an input to type a star system name and an "Add" button that appends it to a Local-Storage-backed array, rendering the current list on load. Add a "Remove" button next to each item (event delegation) that deletes it from both the array and storage.

Portfolio Project Connection

Local Storage is the persistence engine for **Project 04, the Interactive To-Do App** — every add, edit, delete, and complete action must immediately save the updated task array to Local Storage using exactly the load-on-start/save-on-change pattern practiced in this lesson, so tasks survive a page reload.